MACHINE LEARNING WITH SCIKIT-LEARN

Regression Models

05

Open the first black boxes: linear models with regularization, trees, and neighbors — predicting numbers honestly.

Master's Degree Program · Hands-on with Google Colab

AGENDA

Today's agenda

01 **Linear regression, intuitively**
the best straight line

02 **Reading coefficients**
what the model believes

03 **California housing**
our benchmark today

04 **Regularization**
Ridge & Lasso — keeping weights sane

05 **Trees & neighbors for regression**
non-linear patterns

06 **Comparing all of them**
baseline → winner

07 **Pitfalls**
extrapolation, outliers, log targets

BLACK BOX #1

Linear regression: draw the best line



- **One sentence**
- choose the line (or plane) that makes the squared misses smallest
- **Loss = MSE = mean($(y - \hat{y})^2$)**
- no hyperparameters, trains instantly,
- the coefficient story is readable — next slide
- In sklearn: LinearRegression().fit(X, y) — that's it

MATH YOU NEED

The whole model in one line

$$\hat{y} = w_1 \cdot \text{MedInc} + w_2 \cdot \text{HouseAge} + \dots + w_8 \cdot \text{Longitude} + b$$

w (weights / coefficients)

how much the prediction moves when a feature +1

the model's "beliefs" about each column

b (intercept)

prediction when every feature is 0

with scaled features: ≈ the average house value

training

pick w, b minimizing MSE (closed-form or gradient descent — sklearn handles it)

No curve-fitting ritual, no epochs to babysit. One fit() call.

BENCHMARK

California housing — 20,640 block groups

column	meaning	range
MedInc	median income (×10k)	0.5 – 15
HouseAge	median house age	1 – 52
AveRooms, AveOccup…	rooms / occupancy per house	…
Latitude, Longitude	where in California	map coords
y: house value	median price (×100k USD)	0.15 – 5.0

Task: predict a block's median house value from its census features — classic tabular regression.

```
from sklearn.datasets import fetch_california_housing
X, y = fetch_california_housing(return_X_y=True)
```

- **Protocol (from S04)**
- split once: 80/20, random_state 42
- evaluate on the test set only
- baseline first — then models
  - everything scaled inside a Pipeline

INTERPRETATION

Reading the model's mind (scaled features)

feature (standardized)	weight	reads as
MedInc	+0.85	richer block → pricier
AveOccup	−0.04	crowded block → cheaper (slightly)
AveRooms	−0.29	holds other features fixed (!)
Latitude	−0.90	north → cheaper
Longitude	−0.87	east → cheaper
intercept b	+2.07	≈ the average value ($207k)

- **Why coordinates dominate**
- California prices are a geography story (coast!) — linear models combine lat/lon into a tilted plane
- **Careful:**
- weights are PER-STD of the feature
- correlation ≠ causation — description, not policy
- AveRooms negative once income is fixed: big rooms in poor inland blocks

RESULTS

Round 1 — baseline vs linear

model (test set)	MAE	R^2
DummyRegressor(median)	0.874	−0.05
LinearRegression	0.533	0.576

- **In plain words:**
- average miss shrank from $87k to $53k
- **"predict the median" scores $R^2 < 0$**
- worse than predicting the mean — the definition of a floor
- **R^2 0.576 from 8 straight lines — strong start**

Habit

Every regression table in this course starts with the Dummy row.

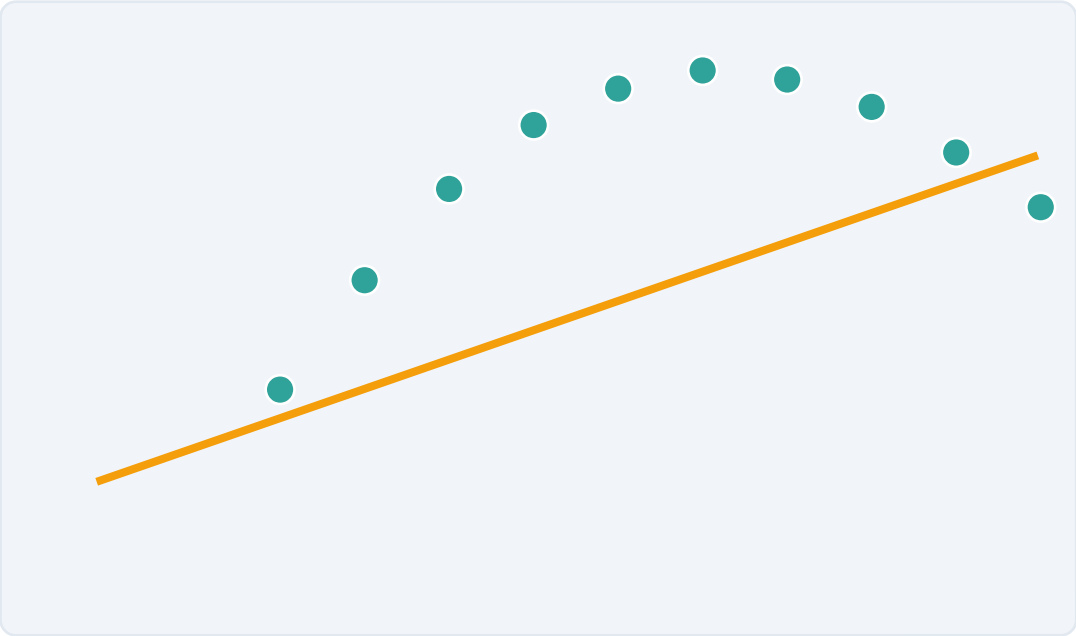
If your fancy model doesn't clearly beat it, the fancy part is decoration.

Why negative R^2?

R^2 compares you to the mean-predictor. Scoring below the mean → below zero. It's a floor, not a bug.

LIMITS

When a straight line isn't enough



one straight line can't follow a bend — it averages the curve away

- **Real prices bend:**
- income helps a lot until it saturates
- distance-to-coast is not linear
- **Diagnosis tool: plot residuals (y − ŷ) vs ŷ**
- a pattern in residuals = the line missed structure
- **Three fixes ahead:**
- polynomial features · trees · neighbors

FIX #1

Polynomial features: bend the plane with linear tools

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
print(X.shape, "->", X_poly.shape)   # (20640, 8) -> (20640, 44)

pipe = Pipeline([("poly", PolynomialFeatures(degree=2)),
                 ("sc",   StandardScaler()),
                 ("m",    LinearRegression())])
```

- **Idea**
- invent x^2, $x_1 \cdot x_2$ … then fit LINEARLY on them
- MedInc^2 captures saturation · Lat×Lon lets the plane curve
- **Still a linear model — in the NEW features**
- **Danger**
- degree 3 on 8 features → 165 columns; degree 5 → 1287
- feature count explodes → overfitting → enter regularization (next)

FIX #1'S BODYGUARD

Ridge & Lasso: penalize wild weights

Ridge (L2) — shrink everything a bit

Loss = MSE + $\alpha \cdot \Sigma w^2$

Big α → all weights pulled toward 0 → smoother model. Keeps ALL features, humbles each one.

Ridge(alpha=1.0)

Lasso (L1) — cut features to zero

Loss = MSE + $\alpha \cdot \Sigma |w|$

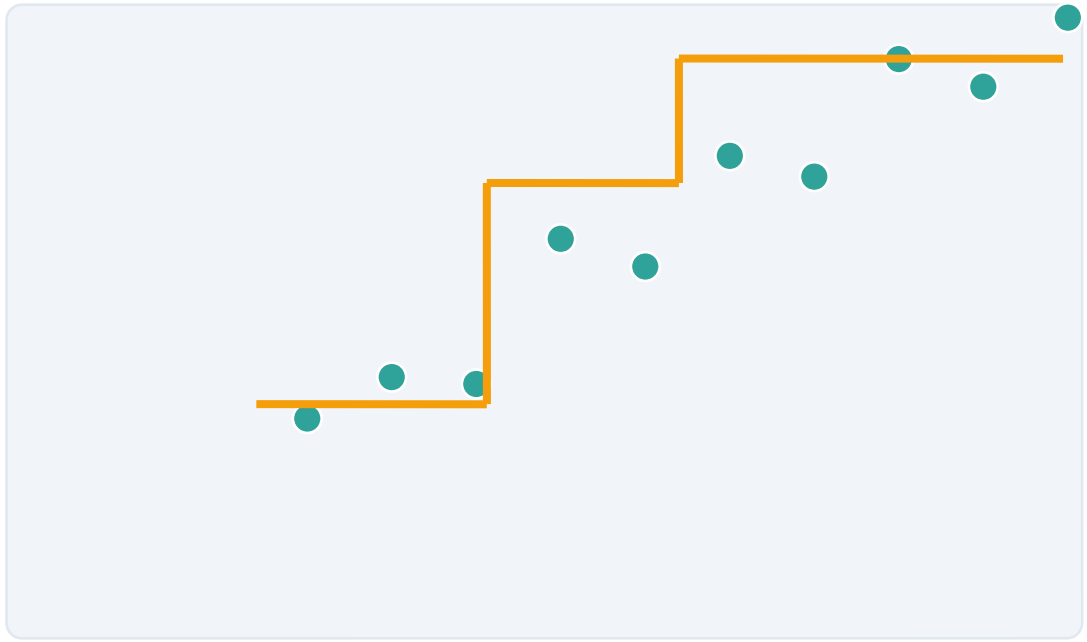
Big α → some weights become exactly 0 → built-in feature selection.

Lasso(alpha=0.1) on housing: test R^2 0.48 — punished TOO hard. α is a hyperparameter (S07 tunes it).

- **On California (8 clean features): Ridge ≈ Linear (R^2 0.576) — nothing to fix**
- Regularization EARNS its keep with MANY / collinear / polynomial features
- **α = the dial between underfit (huge α) and overfit (α→0)**

FIX #2

Decision tree regression: ask, then average



prediction = average value in each region (steps, not slopes)

- **"MedInc < 3.2? → room A : room B" … then predict the room's average price**
- Same tree machinery as classification (full tour in S06)
- **California results:**
- **unrestricted: train R^2 1.000 (!) · test 0.623**
- **max_depth=8: test 0.678, MAE 0.448**
- the S04 overfitting curve, live on housing data

FIX #3

KNN regression: you cost what your neighbors cost

```
KNeighborsRegressor(n_neighbors=10)
# prediction = average value of the 10 most similar block groups
```

model	MAE	R^2
LinearRegression	0.533	0.576
DecisionTree(depth 8)	0.448	0.678
KNN (k=10)	0.440	0.678

- **Neighbors VOTE (classification) or AVERAGE (regression) — same trick as S01**
- **Local memory beats the straight line (0.576 → 0.678) · needs scaling (S03!)**

So which is "best"?

Same R^2 (0.678), different stories:

KNN — "similar places cost similar"
Tree — readable if-then rules
Both beat Linear, but Linear gives the cleanest EXPLANATION.

Prediction ≠ explanation — decide which one your stakeholder needs.

SCOREBOARD

Everything we ran today (test set)

model	MAE	R^2	note
Dummy(median)	0.874	−0.05	floor
LinearRegression	0.533	0.576	readable weights
Ridge(α=1)	0.533	0.576	= linear here: few clean features
Lasso(α=0.1)	0.622	0.481	α too harsh — tune it (S07)
Tree (unrestricted)	0.453	0.624	train R^2 1.000 — memorized
Tree (depth 8)	0.448	0.678	restricted = better
KNN (k=10)	0.440	0.678	needs scaling

MAE in $100k units: 0.44 ≈ "average miss of $44,000" — always translate metrics into money/sentence.

GUIDE

Which regressor when?

situation	start with	strength	watch out
Need explanation / report	LinearRegression	weights = story	only linear patterns
Many/collinear/poly features	Ridge	tames weights	pick α (S07)
Want feature selection	Lasso	zeroes weights	α sensitive
Non-linear, tabular	Tree (depth-limited)	steps & rules	memorizes if free
Local similarity matters	KNN Regressor	no training at all	scale + slow deploy

“Need the best tabular accuracy”? That's ensembles — Session 06.

PITFALLS

Four ways regression fools you

Extrapolation

Trained on incomes up to $150k — asked about $500k. Linear happily predicts outside reality. Trees can't even (they clamp).
Know the data's borders.

Outliers & RMSE

One $5M mansion drags squared error.
RMSE punishes it; MAE shrugs.
Choose which behavior you WANT (S04).

Log targets

Prices span 0.15–5.0 × $100k — skewed.
np.log1p(y) at train, np.expm1 at predict → relative errors, saner model.

R^2 worship

R^2 0.99 on train, 0.62 on test = decoration.
And R^2 compares to the mean-baseline only — always pair with MAE in units.

LAB 05 · DO IT IN GOOGLE COLAB

ทำนายราคาบ้านแคลิฟอร์เนีย — Regression showdown

YOUR MISSION (โจทย์ภาษาไทยใน Colab)

- โหลด California housing แล้วแบ่ง 80/20 (random_state=42) — ใส่ทุกโมเดลใน Pipeline ที่มี StandardScaler
- เริ่มด้วย DummyRegressor แล้ว LinearRegression — รายงาน MAE และ R^2
- ตีความสัมประสิทธิ์ของ LinearRegression: ฟีเจอร์ไหนผลักดันราคาขึ้น/ลงมากที่สุด?
- เทียบ DecisionTree (ไม่จำกัด vs max_depth=8) — ช่องว่าง train/test บอกอะไร?
- โจทย์ชวนคิด: ลอง Ridge กับ alpha หลายค่า (0.01, 1, 100) — สัมประสิทธิ์เปลี่ยนยังไง และอธิบายว่าทำไม Lasso(alpha=0.1) ถึงแย่กว่า Linear

Learning goal

Run the full model zoo on California housing with the S04 protocol: baseline, pipeline, honest comparison, human-readable takeaway.

Vibe coding corner

ลองถาม AI: “แปลง MAE 0.44 ในหน่วย $100k ให้เป็นประโยคภาษาคนสำหรับรายงานผู้บริหาร” — แล้วเขียนสรุป 2 บรรทัดของตัวเองว่าโมเดลไหนควรใช้เพราะอะไร

KEY TAKEAWAYS

1. Linear regression = one weighted sum; weights are the model's beliefs (income +0.85σ, latitude −0.90σ).
2. Ridge shrinks all weights (L2); Lasso zeroes some (L1) — they matter when features are many or engineered.
3. Trees and KNN capture bends (0.576 → 0.678 on housing) but an unrestricted tree memorizes (train R^2 1.0).
4. Always anchor to Dummy (MAE 0.874 / R^2 −0.05) and translate metrics: 0.44 ≈ $44k average miss.
5. Prediction ≠ explanation: pick the model your decision needs.

NEXT

Session 06 — Classification & Ensembles — trees, and committees of trees